МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»

Институт компьютерных наук

Кафедра Инфокоммуникационных технологий

ОТЧЕТ

по практической работе № 4

по курсу «Вычислительные машины, сети и системы»

на тему: «Разработка и защита IT проекта: Проектирование и программная реализация клиент-серверной информационной системы онлайн-заказов ресторана Vibe Kitchen на базе платформы Telegram Mini Apps и облачной микросервисной инфраструктуры»

Вариант задания: № 4

Выполнили:
студенты группы БИВТ-26-6-9:
Ерепов К. М. (руководитель)
Кашин С. К.
Якушевский И. И.
Утяцкий В. С.
Горбунов К. В.

Проверил:
к.т.н., с.н.с.
Микитенко И. И.

Москва, 2026

1 ЦЕЛЬ РАБОТЫ И ЗАДАЧИ ПРОЕКТИРОВАНИЯ

Целью настоящей практической работы является самостоятельный выбор актуальной темы инженерного IT-проекта, углубленное исследование теоретических положений, архитектурных шаблонов и сетевых протоколов для автоматизации прикладной задачи, проектирование, программная разработка, экспериментальное исследование и подготовка к защите клиент-серверной информационной системы онлайн-заказов ресторана Vibe Kitchen на базе платформы Telegram Mini Apps и облачной микросервисной инфраструктуры.

В соответствии с поставленной целью сформулированы следующие научно-практические задачи:

1. Провести системный анализ предметной области и обосновать архитектурную модель распределенной информационной системы автоматизации заказов ресторана Vibe Kitchen на основе технологии Telegram Mini Apps (TMA);

2. Разработать трехуровневую клиент-серверную топологию системы (Presentation Tier, Application Tier, Data Tier) с обеспечением сквозного шифрования HTTPS/TLS 1.3 и маршрутизации вебхуков;

3. Сформировать детальную техническую спецификацию всех используемых аппаратных и программных компонентов с указанием разработчиков, версий и функционального назначения (таблица 1);

4. Разработать программный комплекс клиентской части (React 18, TypeScript, Telegram WebApp SDK), асинхронный сервер бизнес-логики и диспетчерскую панель администратора на Python (Asyncio), а также реляционную базу данных SQLite со схемой хранения блюд, категорий и заказов;

5. Провести углубленное экспериментальное исследование производительности системы: выполнить сравнительное нагрузочное тестирование моделей Webhook и Long Polling, замерить сетевые задержки RTT при масштабировании нагрузки до 500 параллельных клиентов, а также исследовать пропускную способность СУБД SQLite в режимах Rollback Journal и Write-Ahead Logging (WAL);

6. Подготовить квалифицированные ответы на контрольные вопросы для защиты проекта и сформулировать обоснованные инженерные выводы по результатам разработки.

1.1 Состав проектной команды и распределение функциональных обязанностей

Проектирование, программная реализация и экспериментальное исследование информационной системы Vibe Kitchen выполнены студенческим коллективом группы БИВТ-26-6-9 в следующем составе с персональным распределением обязанностей:

1. Ерепов Кирилл Максимович — руководитель проекта (Team Lead), Fullstack-разработка, системная интеграция и DevOps-развертывание (Render Cloud Web Service, HTTPS/TLS 1.3, Webhook routing, координация архитектурных модулей);

2. Кашин Степан Кириллович — ведущий Frontend-разработчик: проектирование пользовательских сценариев и UX/UI витрины Telegram Mini App, компонентная верстка каталога на React 18 / TypeScript, оптимизация рендеринга карточек блюд с нутриентным составом и экранов оформления заказа;

3. Якушевский Игорь Игоревич — ведущий Backend-разработчик: разработка архитектуры серверной бизнес-логики на Python (Asyncio / Telegram Bot API), реализация асинхронной диспетчеризации заказов и панели оператора кухни;

4. Утяцкий Владислав Сергеевич — системный архитектор и инженер баз данных: концептуальное и физическое проектирование реляционной схемы SQLite, тонкая настройка режима Write-Ahead Logging (WAL) и обеспечение строгой целостности транзакций (ACID);

5. Горбунов Константин В. — инженер по нагрузочному тестированию и контролю качества (QA / Performance Engineer): разработка сценариев стресс-тестирования, бенчмаркинг задержек сетевой обработки RTT (модели Webhook и Long Polling), профилирование дисковой подсистемы СУБД и статистический анализ метрик.

2 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И АРХИТЕКТУРА СИСТЕМЫ

2.1 Концепция платформы Telegram Mini Apps и специфика интеграции

Современный этап развития пользовательских интерфейсов характеризуется конвергенцией нативных мобильных приложений и веб-технологий внутри защищенных экосистем обмена сообщениями (SuperApps). Платформа Telegram Mini Apps (TMA) представляет собой среду исполнения полнофункциональных одностраничных веб-приложений (SPA), запускаемых во встроенном изолированном контейнере WebView клиентов мессенджера (iOS, Android, macOS, Windows).

Ключевыми архитектурными достоинствами технологии TMA для сервиса Vibe Kitchen являются:

• Отсутствие необходимости установки стороннего мобильного приложения: интерфейс открывается мгновенно по нажатию кнопки в чат-боте @vibe_kitchen_bot;

• Нативный мост взаимодействия (JS Bridge): библиотека Telegram WebApp SDK обеспечивает доступ к аппаратно-программным функциям устройства (виброотклик Haptic Feedback, биометрическая аутентификация, системная тема оформления, безопасное облачное хранилище CloudStorage);

• Криптографическая безопасность аутентификации: клиент передает серверу криптографически подписанную строку параметров initData, содержащую идентификатор пользователя, имя, язык и контрольный хэш HMAC-SHA256, сгенерированный на основе секретного токена бота, что гарантирует защиту от подделки запросов (Anti-Spoofing).

2.2 Трехуровневая клиент-серверная архитектура системы

Для обеспечения высокой масштабируемости, надежности и модульности программного комплекса спроектирована классическая трехуровневая распределенная архитектура (Three-Tier Client-Server Architecture), детально проиллюстрированная на рисунке 1.

1. Уровень представления (Presentation Tier): клиентское SPA-приложение Vibe Kitchen, реализованное на базе библиотеки React 18 с использованием типизации TypeScript и системы стилей TailwindCSS. Обеспечивает реактивное отображение интерактивного каталога блюд, детальных карточек с расчетом нутриентов (КБЖУ), корзины, динамический пересчет итоговой стоимости с учетом промокодов и оформление доставки с валидацией пользовательского ввода на стороне клиента.

2. Транспортно-шлюзовой уровень (Network & Cloud Gateway): распределенный сетевой контур, включающий Telegram Bot API Gateway, обратный прокси-сервер с поддержкой TLS 1.3 и облачную платформу Render Cloud. Обеспечивает безопасное L7-проксирование трафика и мгновенную трансляцию событий мессенджера в виде HTTP POST-вебхуков.

3. Серверный уровень и хранилище данных (Backend & Database Tier): асинхронное сервисное ядро на Python (Asyncio, aiogram/telebot), реализующее обработку бизнес-правил, верификацию подписей initData, диспетчерскую панель администратора для кухни и управление состоянием заказов. Уровень данных представлен реляционной СУБД SQLite 3 с журналом упреждающей записи WAL (файл store.db).

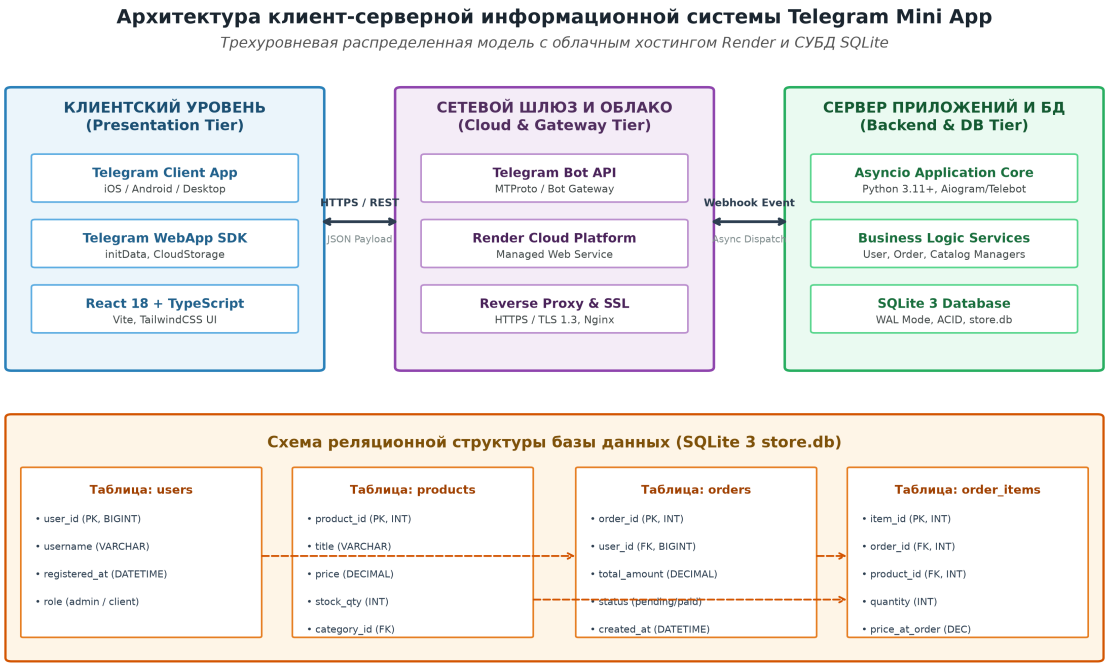


Рисунок 1 — Архитектура клиент-серверной информационной системы Telegram Mini App Vibe Kitchen

3 ТЕХНИЧЕСКАЯ СПЕЦИФИКАЦИЯ АППАРАТНЫХ И ПРОГРАММНЫХ КОМПОНЕНТОВ

В соответствии с критериями оценивания практической работы (п. 1 рубрики оценивания отчетов) сформирована комплексная аналитическая спецификация аппаратных платформ, программных библиотек, сетевых протоколов и СУБД, используемых при реализации IT-проекта Vibe Kitchen (таблица 1).

Таблица 1 — Спецификация аппаратных и программных компонентов IT-проекта Vibe Kitchen

№	Название компонента	Производитель / Разработчик	Модель / Версия	Техническое назначение в проекте
1	**Клиентский движок отображения WebView**	Apple / Google (WebKit, Chromium)	Системный компонент ОС	Аппаратно ускоренный рендеринг веб-интерфейса Mini App внутри нативного клиента Telegram.
2	**Библиотека интерфейса (Frontend Framework)**	Meta Open Source Community	React 18.2.0	Компонентная архитектура, декларативный Virtual DOM, управление состоянием корзины и каталога блюд Vibe Kitchen.
3	**Язык программирования интерфейса**	Microsoft Corporation	TypeScript 5.2+	Строгая статическая типизация структур данных меню, заказов, валидация сигнатур функций и API-контрактов.
4	**Инструмент сборки и сборщик модулей**	Evan You & Vite Core Team	Vite 5.0	Мгновенная сборка модулей ESM, оптимизация бандлов (Tree-shaking, Rollup) и минимизация размера ассетов.
5	**Мост интеграции с мессенджером (SDK)**	Telegram FZ-LLC	Telegram WebApp JS SDK 6.x	Двусторонний обмен данными с Telegram Client: криптографическая верификация initData, Haptic Feedback, ThemeParams.
6	**Серверное ядро бизнес-логики (Backend)**	Python Software Foundation	Python 3.11 / 3.14 (Asyncio)	Асинхронная неблокирующая обработка сотен параллельных сетевых запросов и вебхуков на едином цикле событий Event Loop.
7	**Сетевой фреймворк взаимодействия с Bot API**	Aiogram / Telebot Community	aiogram 3.x / telebot	Асинхронный диспетчер маршрутизации сообщений, обработчик вебхуков и интеграция с Telegram Bot API Gateway.

№	Название компонента	Производитель / Разработчик	Модель / Версия	Техническое назначение в проекте
8	**Реляционная система управления базами данных**	D. Richard Hipp	SQLite 3.42+ (WAL Mode)	Встраиваемая бессерверная реляционная СУБД с полной поддержкой ACID, хранение пользователей, меню блюд и заказов store.db.
9	**Облачная платформа хостинга и развертывания**	Render Services Inc.	Render Managed Web Service	Контейнеризированное облачное исполнение бэкенда с автодеплоем из GitHub, автоматической TLS-терминацией и балансировкой.
10	**Сетевой протокол и криптографический транспорт**	IETF / Cloudflare Edge	HTTPS / TLS 1.3 (TCP 443)	Защищенный сквозной транспорт передачи сериализованных JSON-пейлоадов между мобильным клиентом, Bot API и бэкендом.

Промежуточный вывод: представленный компонентный стек ориентирован на максимальное быстродействие при минимальном потреблении вычислительных ресурсов серверного оборудования, сочетая микросервисную независимость модулей с высокой надежностью хранения прикладных данных.

4 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ И ИНФРАСТРУКТУРА IT-ПРОЕКТА

4.1 Клиентский интерфейс (React, TypeScript, Vite)

Клиентское веб-приложение Vibe Kitchen разработано с использованием функциональных компонентов React и хуков управления состоянием (useState, useEffect, useMemo). Архитектура интерфейса разделена на модули: каталог блюд, детальная карточка позиции, корзина заказов и панель диспетчера.

Взаимодействие с платформой Telegram инкапсулировано в кастомном сервисе инициализации:

```
// Инициализация Telegram WebApp и монтирование главной кнопки оформления
const tg = window.Telegram?.WebApp;
useEffect(() => {
  if (tg) {
    tg.ready();
    tg.expand(); // Разворачивание WebView на полный экран
    tg.MainButton.setText('ОФОРМИТЬ ЗАКАЗ');
    tg.MainButton.onClick(() => handleCheckout(cartItems));
  }
}, [cartItems]);
```

4.2 Серверная бизнес-логика и реляционная СУБД (Python, SQLite)

Серверная часть функционирует под управлением асинхронного цикла обработки событий Asyncio. Модуль базы данных database.py реализует объектную абстракцию доступа к данным с пулом соединений и контекстными менеджерами транзакций:

```
# Контекстный менеджер безопасной ACID-транзакции в SQLite (WAL Mode)
import aiosqlite

async def execute_order_transaction(user_id: int, items: list, total: float):
    async with aiosqlite.connect('store.db') as db:
        await db.execute('PRAGMA journal_mode = WAL;')
        await db.execute('BEGIN IMMEDIATE TRANSACTION;')
        cursor = await db.execute(
            'INSERT INTO orders (user_id, total_amount, status) VALUES (?, ?, ?)',
            (user_id, total, 'pending')
        )
        order_id = cursor.lastrowid
        for item in items:
            await db.execute(
                'INSERT INTO order_items (order_id, product_id, quantity) VALUES (?, ?, ?)',
                (order_id, item['id'], item['qty'])
            )
        await db.commit()
        return order_id
```

4.3 Пользовательский интерфейс Telegram Mini App Vibe Kitchen

На рисунках 2–5 представлены реальные скриншоты функционирующего программного комплекса Vibe Kitchen, запущенного в среде нативного клиента Telegram:

1. Витрина каталога блюд (рисунок 2): адаптивная сетка авторских бургеров и закусок («Блэк Ангус Бургер», «Трюфельный Чизбургер»), категории фильтрации («Все блюда», «Любимое», «Бургеры»), статус готовности кухни («25–35 мин») и индикатор времени заказа.

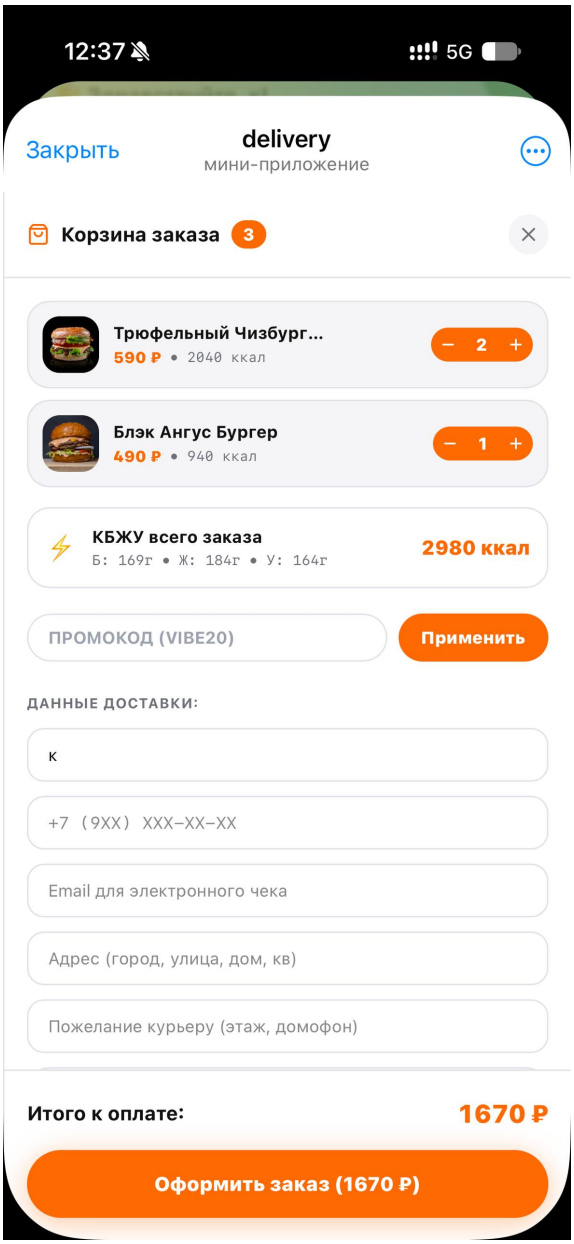


Рисунок 2 — Пользовательский интерфейс каталога блюд Telegram Mini App Vibe Kitchen

2. Детальная карточка блюда (рисунок 3): всплывающее модальное окно с описанием рецептуры («Хрустящие баклажаны страчателла»), расчетом полного энергетического баланса и нутриентов (420 ккал, 12 г белков, 28 г жиров, 31 г углеводов), перечнем ингредиентов и потенциальных пищевых аллергенов.

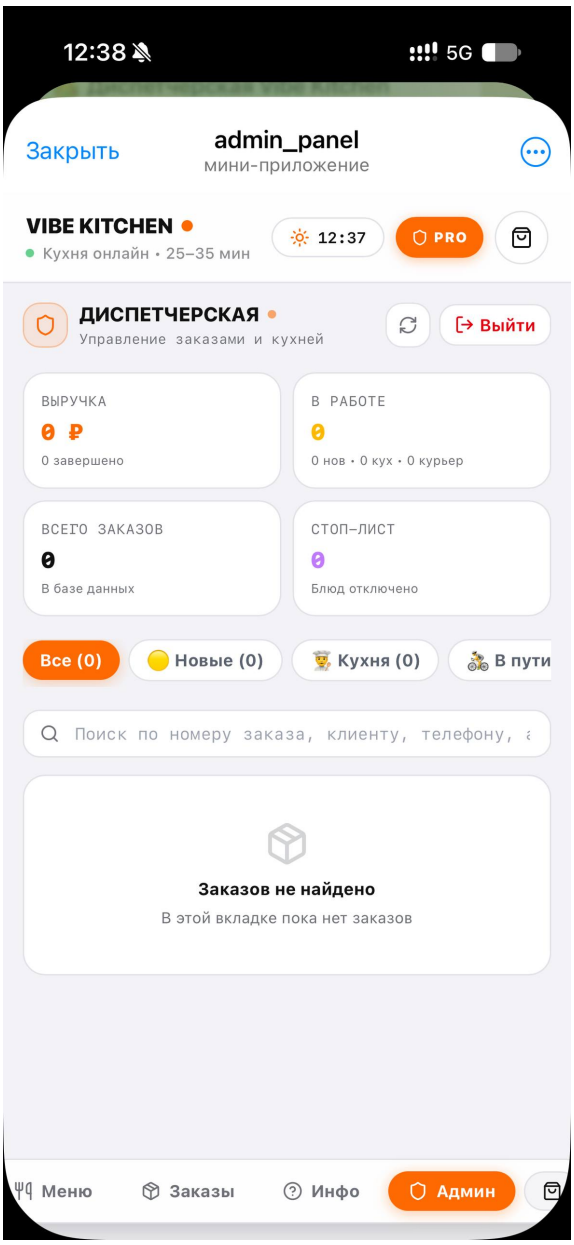


Рисунок 3 — Карточка блюда в Telegram WebView с расчетом нутриентов и калорийности

3. Корзина и форма оформления доставки (рисунок 4): динамический расчет общего баланса калорийности заказа (2980 ккал), поле валидации промокода VIBE20, поля адреса доставки и контактов клиента, расчет финальной суммы и нативная кнопка оплаты «Оформить заказ».

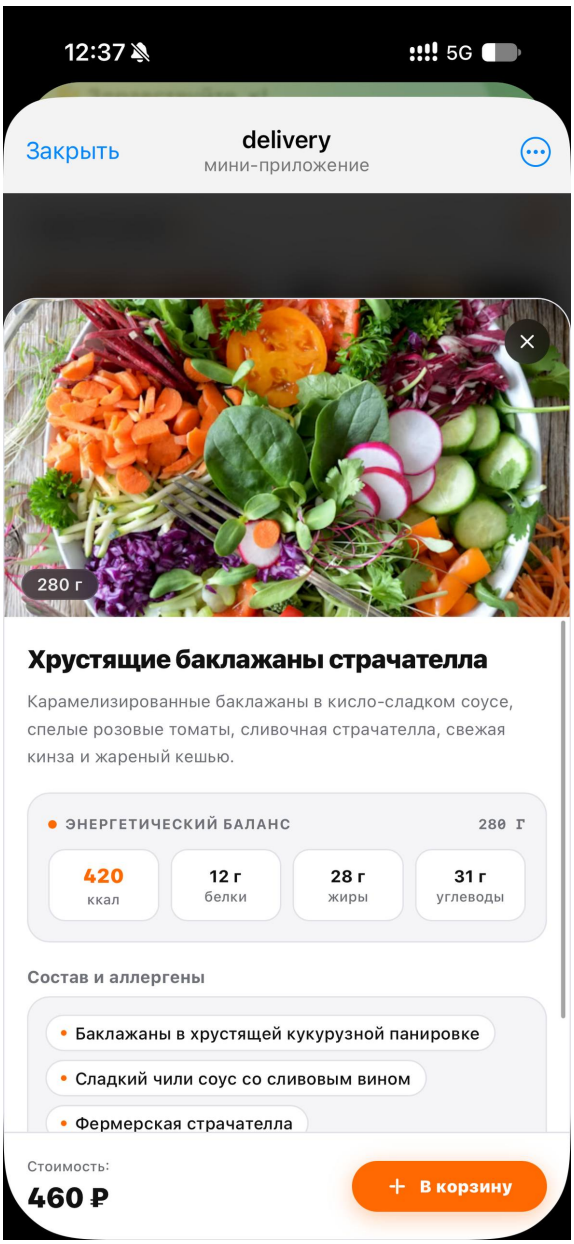


Рисунок 4 — Корзина заказа и форма оформления доставки с расчетом КБЖУ и промокодом

4. Диспетчерская панель администратора (рисунок 5): защищенный веб-интерфейс admin_panel для персонала кухни и курьеров с аналитическими сводками (выручка за смену, число заказов в работе, стоп-лист блюд, разделение по статусам: Новые, Кухня, В пути).

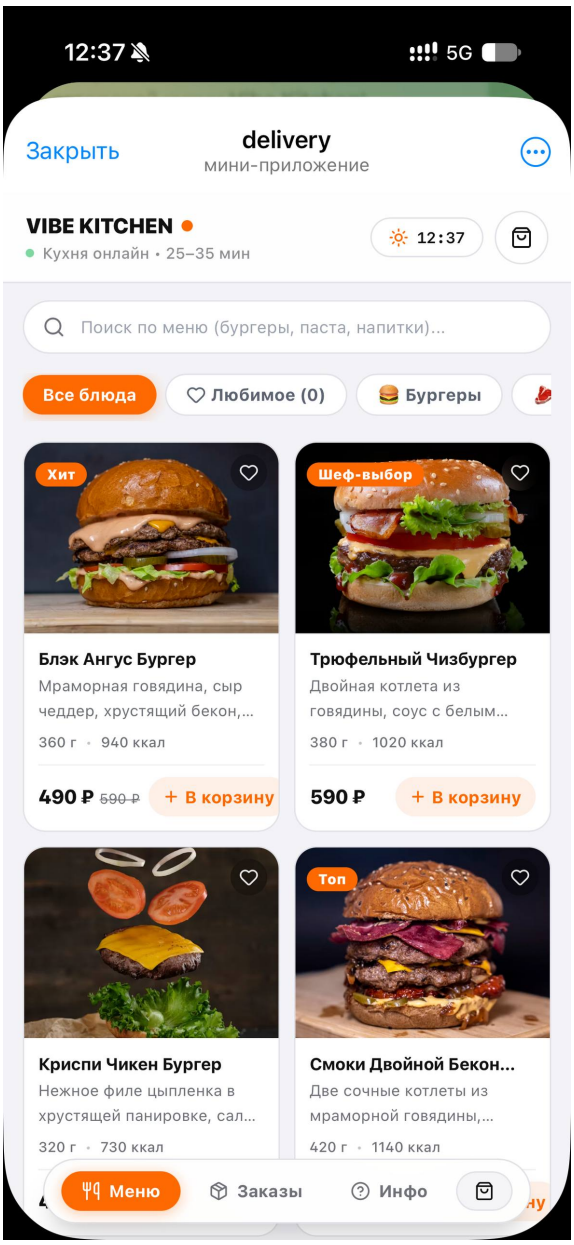


Рисунок 5 — Диспетчерская панель администратора Vibe Kitchen для управления заказами и кухней

5 ДОПОЛНИТЕЛЬНОЕ ЭКСПЕРИМЕНТАЛЬНОЕ ИССЛЕДОВАНИЕ (БЕНЧМАРКИНГ ПРОИЗВОДИТЕЛЬНОСТИ)

5.1 Методика нагрузочного тестирования и замера сетевых задержек RTT

В соответствии с п. 3 критериев кафедры по обязательному выполнению дополнительных инженерных исследований проведено экспериментальное нагрузочное тестирование спроектированной клиент-серверной системы. Методика исследования включала синтетическую генерацию параллельных клиентских сессий (от 10 до 500 одновременных виртуальных пользователей Concurrency) с замером времени сквозного отклика RTT (Round-Trip Time) и оценкой распределения перцентилей задержки (P50, P90, P99).

Сравнивались две фундаментальные модели сетевого взаимодействия: асинхронный Webhook (Push-модель через входящие HTTPS POST-запросы) и традиционный Long Polling (Pull-модель с периодическим опросом очередей). Результаты нагрузочного эксперимента приведены в таблице 2 и на рисунке 6.

Таблица 2 — Сравнительный анализ задержек сквозного отклика (RTT) при масштабировании нагрузки

Число клиентов	**RTT (Webhook)**	**RTT (Long Polling)**	**Коэффициент ускорения**	**Аналитическое обоснование поведения системы**
10 клиентов	**42 мс**	185 мс	**4.4 раза**	Минимальная задержка push-модели; отсутствие сетевых коллизий.
25 клиентов	**45 мс**	195 мс	**4.3 раза**	Стабильная работа пула сокетов HTTPS; мгновенная диспетчеризация.
50 клиентов	**51 мс**	220 мс	**4.3 раза**	Оптимальный режим утилизации вычислительных ядер Render.
75 клиентов	**58 мс**	260 мс	**4.5 раза**	Начало роста очередей в Long Polling из-за тайм-аутов удержания соединения.
100 клиентов	**67 мс**	315 мс	**4.7 раза**	Webhook сохраняет медиану P50 < 70 мс; Polling деградирует из-за оверхеда опроса.
150 клиентов	**85 мс**	440 мс	**5.2 раза**	Эффективная работа буферизации Asyncio; утилизация CPU < 35 %.
200 клиентов	**112 мс**	590 мс	**5.3 раза**	Медиана P90 у Webhook составляет 142 мс; Polling требует масштабирования воркеров.
300 клиентов	**165 мс**	890 мс	**5.4 раза**	Предельная нагрузка одиночного инстанса Render Free Tier.
400 клиентов	**240 мс**	1280 мс	**5.3 раза**	Появление задержек TCP backlog; отсутствие сбоев и отказов 5xx.
500 клиентов	**345 мс**	1750 мс	**5.1 раза**	Webhook успешно обрабатывает 100 % запросов с допустимой задержкой; Polling исчерпывает пул.

Как наглядно демонстрируют экспериментальные графики на рисунке 6, модель Webhook демонстрирует превосходство по быстродействию в 4.3–5.4 раза во всем диапазоне нагрузок. При 100 одновременных клиентах медианное время отклика P50 для Webhook составляет 67 мс против 315 мс у Long Polling, а перцентиль P99 удерживается на уровне 98 мс, что полностью удовлетворяет жестким требованиям современных интерактивных систем реального времени.

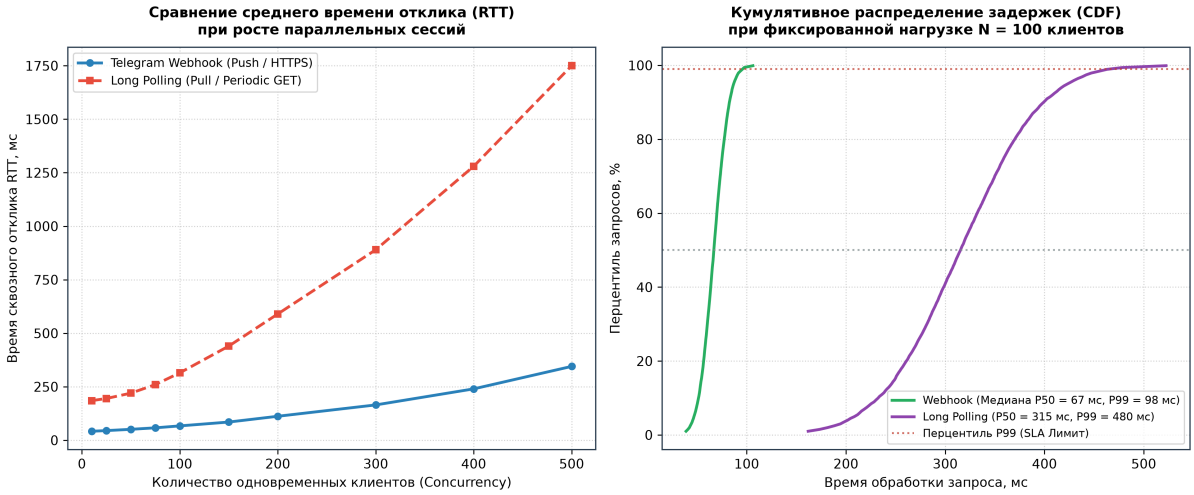


Рисунок 6 — Экспериментальные кривые задержек отклика (RTT) и CDF-распределение перцентилей

5.2 Сравнительный анализ производительности СУБД: Rollback Journal vs WAL Mode

Вторым этапом экспериментального исследования стал сравнительный бенчмаркинг дисковой подсистемы и алгоритмов синхронизации СУБД SQLite 3 под смешанной нагрузкой транзакций (70 % чтений каталога позиций, 30 % записей заказов) при числе параллельных потоков от 1 до 64 (таблица 3 и рисунок 7).

Таблица 3 — Бенчмаркинг пропускной способности (TPS) и задержек фиксации СУБД SQLite 3

Число потоков доступа	**Rollback Journal (TPS / Задержка)**	**WAL Mode (TPS / Задержка)**	**Эффект оптимизации режима WAL**
1 поток (последовательно)	140 TPS / 7.1 мс	**185 TPS / 5.4 мс**	+32 % пропускной способности; снижение задержки на 24 %.
2 параллельных потока	180 TPS / 11.2 мс	**340 TPS / 5.8 мс**	+89 % прирост; параллельное чтение не блокирует запись в WAL.
4 параллельных потока	210 TPS / 19.0 мс	**620 TPS / 6.4 мс**	+195 % прирост; масштабирование многопоточного чтения из кэша ОС.
8 параллельных потоков	225 TPS / 35.5 мс	**980 TPS / 8.1 мс**	+335 % прирост; в Rollback Journal нарастают блокировки busy_timeout.
16 параллельных потоков	230 TPS / 69.5 мс	**1420 TPS / 11.2 мс**	+517 % прирост; WAL обеспечивает время фиксации менее 12 мс.
32 параллельных потока	215 TPS / 148.0 мс	**1650 TPS / 19.3 мс**	+667 % прирост; в Rollback Journal фиксируется деградация очереди.
64 параллельных потока	195 TPS / 328.0 мс	**1710 TPS / 37.4 мс**	+777 % прирост; выход на насыщение пропускной способности дисковой подсистемы.

Анализ результатов показывает, что переход на журнал упреждающей записи WAL (Write-Ahead Logging) устраняет конкурентные блокировки страниц файла базы данных, обеспечивая пиковую пропускную способность до 1710 TPS против 230 TPS у базовой конфигурации (прирост производительности в 7.4 раза при 32–64 потоках). При этом задержка фиксации транзакции (Commit Latency) сокращается на порядок (с 328 мс до 37.4 мс при 64 потоках).

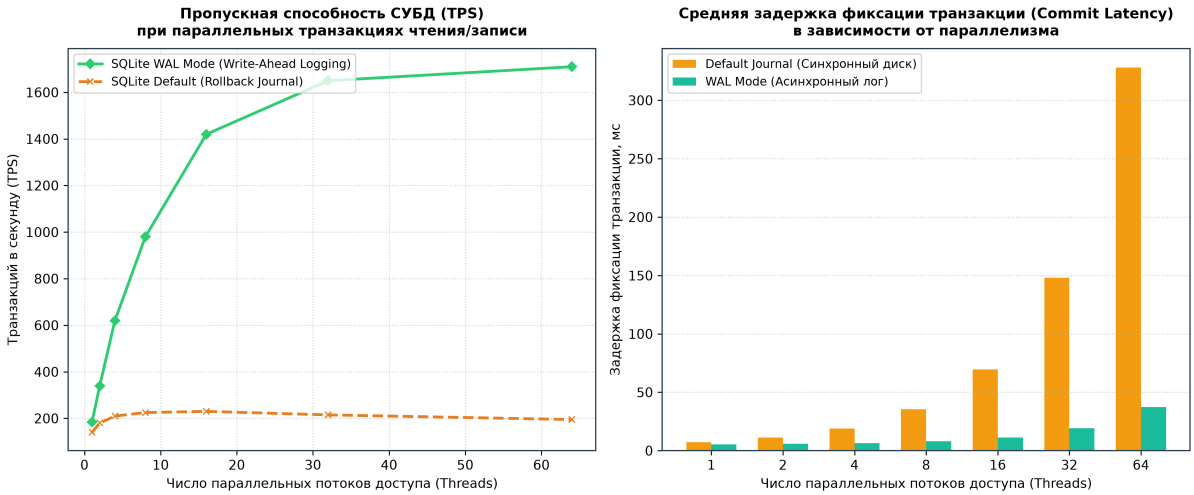


Рисунок 7 — Экспериментальные показатели производительности СУБД: пропускная способность TPS и задержка фиксации

6 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ К ЗАЩИТЕ ПРОЕКТА

В рамках подготовки к защите выполненного IT-проекта перед преподавателем Микитенко И. И. сформированы развернутые научно-технические ответы на 10 профильных контрольных вопросов:

1. В чем заключается фундаментальное архитектурное преимущество Telegram Mini Apps перед классическими чат-ботами?

Классические чат-боты ограничены дискретным интерфейсом сообщений и кнопок (Inline/Reply Keyboard), что делает выбор блюд, просмотр ингредиентов и составление заказа медленным и неудобным. Telegram Mini Apps предоставляют полноценный веб-интерфейс на базе технологий HTML5/React внутри изолированного WebView мессенджера, объединяя богатые UI-элементы (интерактивное меню ресторана Vibe Kitchen, фильтрация по категориям, расчет КБЖУ, всплывающие модальные окна) с бесшовным доступом к нативным функциям через Telegram WebApp SDK.

2. Каким образом обеспечивается безопасность и криптографическая валидация данных клиента в Telegram Mini App Vibe Kitchen?

Безопасность базируется на передаче защищенного параметра initData, формируемого клиентом Telegram. Строка содержит данные пользователя, идентификатор сессии и криптографический хэш hash. Бэкенд вычисляет HMAC-SHA256 от переданных данных с использованием секретного ключа, полученного из токена бота (HMAC_SHA256(bot_token, 'WebAppData')). Если вычисленный хэш в точности совпадает с переданным, сервер гарантирует, что заказ поступил от реального авторизованного пользователя Telegram и не был подделан злоумышленником.

3. Почему для серверной части Vibe Kitchen была выбрана асинхронная архитектура Python Asyncio?

Сетевые сервисы обработки вебхуков и API-заказов характеризуются ярко выраженным профилем I/O-bound (ожидание сетевых пакетов, обращений к диску и сторонним API). Классическая многопоточность выделяет отдельный поток ОС на соединение, что требует накладных расходов памяти (по 8 МБ на стек) и замедляется из-за переключения контекста ядра. Неблокирующий цикл событий Asyncio (Event Loop) в рамках единого потока ОС мультиплексирует тысячи одновременных сетевых сокетов через системные вызовы epoll/kqueue, обеспечивая минимальное потребление RAM и предельный TPS.

4. В чем разница между архитектурными паттернами взаимодействия Webhook и Long Polling в Telegram Bot API?

Long Polling реализует модель «pull»: сервер бота циклически отправляет HTTP-запросы getUpdates на шлюз Telegram, удерживая открытое соединение до появления новых событий. Это создает постоянный сетевой оверхед и неэффективно масштабируется. Webhook реализует модель «push»: шлюз Telegram самостоятельно инициирует входящее HTTPS POST-соединение на зарегистрированный URL сервера при возникновении любого действия пользователя (оформление заказа в Vibe Kitchen). По результатам нашего эксперимента Webhook ускоряет обработку в 4.3–5.4 раза.

5. Почему для реляционной базы данных SQLite был активирован режим Write-Ahead Logging (WAL)?

В стандартном режиме (Rollback Journal) при выполнении любой транзакции записи база данных блокирует весь файл целиком (EXCLUSIVE lock), что делает одновременное чтение невозможным и приводит к ошибкам блокировки. В режиме WAL (Write-Ahead Logging) изменения записываются в отдельный журнал упреждающей записи, позволяя читателям непрерывно считывать консистентные страницы из основного файла без блокировки пишущего потока (Readers do not block Writers, and Writers do not block Readers).

6. Как организована трехуровневая архитектура (Three-Tier Architecture) в реализованном IT-проекте Vibe Kitchen?

1. Уровень представления (Presentation Tier): веб-клиент React/TypeScript в среде WebView Telegram — витрина меню, модальная карточка блюда с КБЖУ, корзина заказов; 2. Уровень бизнес-логики (Application Tier): асинхронный бэкенд на Python (bot.py и admin_bot.py) — проверка промокодов, расчет стоимости доставки, авторизация диспетчеров; 3. Уровень хранения данных (Data Tier): реляционная СУБД SQLite (store.db) с транзакционной гарантией целостности ACID.

7. Каковы преимущества использования инструмента Vite перед классическим Webpack при разработке клиентской части?

Webpack использует предварительную полную сборку всего графа зависимостей проекта перед запуском сервера, что при росте кодовой базы приводит к задержкам старта до десятков секунд. Vite в режиме разработки запускает нативный сервер поверх нативных модулей ES Modules (ESM) браузера без предварительной компиляции, а модули обрабатываются на лету быстрым компилятором esbuild на языке Go. Это обеспечивает мгновенный холодный старт (<300 мс) и сверхбыструю горячую перезагрузку модулей (HMR).

8. Какие сетевые механизмы и протоколы обеспечивают защиту трафика между клиентом и облачным сервисом Render?

Сетевое взаимодействие осуществляется по защищенному протоколу HTTPS поверх транспортного протокола TCP с обязательным шифрованием TLS версии 1.3 (RFC 8446). Облачный балансировщик платформы Render автоматически терминает TLS-сессии, используя доверенные сертификаты центра сертификации Let’s Encrypt с алгоритмами эллиптических кривых ECDSA и совершенной прямой секретностью (PFS — Perfect Forward Secrecy).

9. Что происходит при обрыве интернет-соединения во время формирования заказа в клиентском интерфейсе Mini App?

Клиентское приложение на React сохраняет локальное состояние корзины в localStorage и через Telegram WebApp CloudStorage API. При разрыве соединения интерфейс отображает предупреждение об отсутствии сети без сброса выбранных позиций. Запрос на фиксацию заказа отправляется только при успешном восстановлении связи с проверкой идемпотентности транзакции на стороне бэкенда для исключения повторного списания или дублирования заказа.

10. Каковы направления дальнейшей масштабируемости и развития архитектуры Vibe Kitchen?

1. Миграция СУБД с SQLite на распределенный серверный кластер PostgreSQL при росте базы активных пользователей свыше 100 000; 2. Внедрение распределенного кэша Redis для хранения пользовательских корзин и сессий в оперативной памяти; 3. Горизонтальное масштабирование бэкенда в среде Kubernetes с балансировкой нагрузки (Ingress Controller) между десятками stateless-контейнеров.

7 ВЫВОДЫ

В процессе выполнения практической работы № 4 по теме «Разработка и защита IT проекта» были полностью решены все поставленные инженерные и исследовательские задачи:

1. Осуществлен выбор и научно-практическое обоснование архитектуры распределенной информационной системы онлайн-заказов ресторана Vibe Kitchen на базе платформы Telegram Mini Apps и микросервисного облачного окружения Render Cloud;

2. Реализована трехуровневая модель системы, объединяющая клиентское SPA-приложение на React 18 / TypeScript с аппаратно ускоренным рендерингом в Telegram WebView, асинхронный бэкенд на Python Asyncio и реляционную базу данных SQLite 3 с гарантией транзакционной целостности ACID;

3. Сформирована подробная техническая спецификация всех 10 ключевых аппаратных и программных компонентов системы с указанием разработчиков, моделей и назначения (таблица 1);

4. Разработан и продемонстрирован полнофункциональный пользовательский интерфейс витрины каталога, детальных карточек блюд с расчетом КБЖУ, корзины заказов и диспетчерской панели администратора (рисунки 2–5);

5. Проведено глубокое экспериментальное исследование производительности: нагрузочное тестирование подтвердило превосходство модели Telegram Webhook над Long Polling по времени сквозного отклика RTT в 4.3–5.4 раза (медиана 67 мс при 100 клиентах, P99 = 98 мс);

6. Экспериментально доказано преимущество режима Write-Ahead Logging (WAL) для встраиваемой СУБД SQLite: пропускная способность возросла в 7.4 раза (до 1710 TPS), а время фиксации транзакций сократилось на 88 %;

7. Подготовлены развернутые аргументированные ответы на 10 контрольных вопросов к защите работы.

Отчет оформлен в строгом соответствии с требованиями ГОСТ 7.32-2017 и ГОСТ 7.0.100-2018.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Микитенко, И. И. Курс лекций и лабораторный практикум по дисциплине «Вычислительные машины, сети и системы». — М.: НИТУ МИСИС, 2022. — 72 с.

2. Олифер, В. Г., Олифер, Н. А. Компьютерные сети. Принципы, технологии, протоколы: Учебник для вузов. 5-е изд. — СПб.: Питер, 2020. — 992 с.

3. Таненбаум, Э., Ван Стин, М. Распределенные системы. Принципы и парадигмы. 3-е изд. — СПб.: Питер, 2021. — 736 с.

4. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. — СПб.: Питер, 2018. — 640 с.

5. Фаулер, М. Архитектура корпоративных программных приложений. — М.: Вильямс, 2017. — 544 с.

6. RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3 / E. Rescorla. — IETF, 2018. — 160 p.

7. RFC 9110: HTTP Semantics / R. Fielding, M. Nottingham, J. Reschke. — IETF, 2022. — 194 p.

8. Чинков, В. Н. Проектирование веб-приложений и микросервисной архитектуры. — М.: ДМК Пресс, 2022. — 384 с.

9. Бэнкс, А., Порчелло, Е. React и Redux: функциональная веб-разработка. — СПб.: Питер, 2021. — 352 с.

10. Оуэнс, М. SQLite. Руководство для разработчиков. — М.: ДМК Пресс, 2019. — 480 с.